

NCTU-EE IC Design LAB – 2025Fall

Lab11 Exercise: Innovus - From RTL to GDSII

In this exercise, you are going to perform the APR steps to the design: GTE

1. Memory lef files modification

1. Modify all the Memory.lef file
 - Change “core” to “core_5040”
 - Change “ME1” to “metal1” and so on
 - Change “VI1” to “via”
 - Change “VI2” to “via2” and so on

```
NAMESCASESENSITIVE ON ;
MACRO MEM_IMAGE
CLASS BLOCK ;
FOREIGN MEM_IMAGE 0.000 0.000 ;
ORIGIN 0.000 0.000 ;
SIZE 994.480 BY 176.400 ;
SYMMETRY x y r90 ;
SITE core ;
PIN VCC
DIRECTION INOUT ;
USE POWER ;
SHAPE ABUTMENT ;
PORT
LAYER ME4 ;
RECT 993.360 164.980 994.480 168.220 ;
LAYER ME3 ;
RECT 993.360 164.980 994.480 168.220 ;
LAYER ME2 ;
RECT 993.360 164.980 994.480 168.220 ;
LAYER ME1 ;
RECT 993.360 164.980 994.480 168.220 ;
END
LAYER VI1 ;
RECT 0.000 0.140 994.480 176.400 ;
LAYER VI2 ;
RECT 0.000 0.140 994.480 176.400 ;
LAYER VI3 ;
RECT 0.000 0.140 994.480 176.400 ;
```

2. Data Preparation

2. Change the directory to **05_APR**
3. Prepare chip design netlist
 - Open **CHIP_SHELL.v** and complete the definition of pad cells
 - ✧ The top module name is **CHIP**
 - ✧ This CHIP contains the module **GTE**; and **I/O**, **I/O power**, **core power pad**
 - ✧ Please calculate how many I/O power pads and core power pads you need, and complete the netlist of pad cells
 - After defining the pad cells, please run “./00_combine” to combine the GTE_SYN.v with CHIP_SHELL.v to be CHIP_SYN.v

```
/Lab11_2025F/05_APR]$ ./00_combine
```

4. Prepare I/O pad location file:
 - Please see **CHIP.io** to know how to assign I/O pad location.
 - Open **CHIP.io** and complete the I/O pad location assignment according to the netlist of pad cells in **CHIP_SYN.v**
5. Prepare timing/driving/loading constraint file **CHIP.sdc** and **CHIP_cts.sdc**
 - **CHIP.sdc**:
 - ✧ `cp ../02_SYN/Netlist/GTE_SYN.sdc CHIP.sdc`
 - ✧ Comment out the `set_wire_load_mode` & `set_wire_load_model` instruction

```
#set_wire_load_mode top
#set_wire_load_model -name G5K -library fsa0m_a_generic_core_ss1p62v125c
```
 - **CHIP_cts.sdc**:
 - ✧ TA has provided this file in 05_APR. **You should change the**

desired **cycle time**, and the **input & output delay** should be set as **half of the cycle time** in case there are timing violations in post-layout simulation. (You can modify the cycle time before CTS)

```
#####
# You should modify your desired cycle time for post-layout simulation (06_POST)
# input / output delay should be half of cycle time, or you might get failure in this lab
#####
create_clock [get_ports clk] -period 20.0 -waveform {0 10.0}
set_input_delay 10.0 -clock clk [remove_from_collection [all_inputs] [get_ports clk]]
set_output_delay 10.0 -clock clk [all_outputs]
```

6. Prepare linked library files:

- Timing libraries (LIB)
 - ✧ **fsa0m_a_generic_core_ss1p62v125c.lib**
 - ✧ **fsa0m_a_generic_core_ff1p98vm40c.lib**
 - ✧ **fsa0m_a_t33_generic_io_ss1p62v125c.lib**
 - ✧ **fsa0m_a_t33_generic_io_ff1p98vm40c.lib**
 - ✧ **Memory_WC.lib**
 - ✧ **Memory_BC.lib**
- Physical libraries (LEF)
 - ✧ **header6_V55_20ka_cic.lef**
 - ✧ **fsa0m_a_generic_core.lef**
 - ✧ **FSA0M_A_GENERIC_CORE_ANT_V55.lef**
 - ✧ **fsa0m_a_t33_generic_io.lef**
 - ✧ **FSA0M_A_T33_GENERIC_IO_ANT_V55.lef**
 - ✧ **BONDPAD.lef**
 - ✧ **Memory.lef**
- RC extraction table/files (RC)
 - ✧ **u18_Faraday.CapTbl**
 - ✧ **icecaps.tch**
- SI libraries (CeltIC)
 - ✧ **u18_ss.cdb**
 - ✧ **u18_ff.cdb**
- GDSII layout (Will not stream out in this course)
 - ✧ **u18.gds2**
 - ✧ **u18io3v5v_6lm.gds2**
 - ✧ **Memory.gds2** (not provided in this compiler version)

3. Reading Cell Library information and Netlist for APR

7. If you are running on ee servers, type below command on terminal to set OA_HOME.

```
% setenv OA_HOME
```

```
/RAID2/cad/cadence/INNOVUS/INNOVUS_20.15.000/share/oa
```

```
@ee31[~/Lab11/05_APR]$ setenv OA_HOME /RAID2/cad/cadence/INNOVUS/INNOVUS_20.15.000/share/oa
```

8. Start Innovus in the directory **05_APR**

% innovus (no &, do NOT use background execution)

```
@ee31[~/Lab11/05_APR]$ innovus
```

9. Set unquify global variable to 1 (this can give an unduplicated name to each cells)

innovus > set init_design_uniquify 1

```
innovus 1> set init_design_uniquify 1
```

10. Change design mode to 0.18um process (initial setting is 90 nm process)

innovus > setDesignMode -process 180

```
innovus 2> setDesignMode -process 180
```

11. Skip error messages like the following figure when reading lef files

innovus > suppressMessage TECHLIB 1318

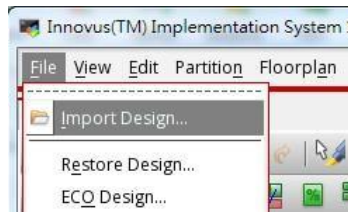
innovus > suppressMessage ENCEXT-2799

```
innovus 3> suppressMessage TECHLIB 1318
innovus 4> suppressMessage EXCEXT-2799
```

```
**ERROR: (TECHLIB-1318):      All the table values in the 'rise_transition' group are within '0
.000010' of each other. (File /home/RAID2/COURSE/iclab/iclabta01/umc018/Lib/umc18io3v5v_slow.lib,
Line 682)
**ERROR: (TECHLIB-1318):      All the table values in the 'fall_transition' group are within '0
.000010' of each other. (File /home/RAID2/COURSE/iclab/iclabta01/umc018/Lib/umc18io3v5v_slow.lib,
Line 690)
**ERROR: (TECHLIB-1318):      All the table values in the 'rise_transition' group are within '0
.000010' of each other. (File /home/RAID2/COURSE/iclab/iclabta01/umc018/Lib/umc18io3v5v_slow.lib,
Line 782)
```

This error message occurs since innovus assumes the values of rise / fall transition time should differ larger than 0.00001 ns in **the table** depending on the supply Voltage or the Temperature in **.lef file**. The error message can be ignored if the given .lef file is ensured to be correct. (Note: only suppress error when the reason is clearly verified instead of suppressing all errors)

12. In innovus menu, open **File → Import Design**



13. Fill the following field:

- Netlist
 - ✧ ◆ Verilog
 - ✧ Files: CHIP_SYN.v
 - ✧ Top Cell: ◆ By User : CHIP
- Technology / Physical Libraries
 - ✧ ◆ LEF Files
 - ✧ header6_V55_20ka_cic.lef

- ✧ fsa0m_a_generic_core.lef
 - ✧ FSA0M_A_GENERIC_CORE_ANT_V55.lef
 - ✧ fsa0m_a_t33_generic_io.lef
 - ✧ FSA0M_A_T33_GENERIC_IO_ANT_V55.lef
 - ✧ BONDPAD.lef
 - ✧ Memory.lef
- You need to add these files based on this sequence,
or you may fail to do the next step.

Note that (1) the standard cell lef file should be put in the first place since it contains the major metals / vias / polys technology definitions. Also (2) if there are antenna lef files, they should be put after the corresponded lef files without describing antenna layers. For example: umc18_6lm.lef should be put in the first place; umc18_6lm_antenna.lef should be put after umc18_6lm.lef.

- Floorplan
 - ✧ IO Assignment Files: CHIP.io
- Power
 - ✧ Power Nets: VCC
 - ✧ Ground Nets: GND
- Analysis Configuration

Press “Create Analysis Configuration” tab

- ✧ Double click Library Sets and include the max and min delay library:

Max delay:

Name: lib_max

Timing Library:

fsa0m_a_generic_core_ss1p62v125c.lib

fsa0m_a_t33_generic_io_ss1p62v125c.lib

Memory_WC.lib

SI Library: u18_ss.cdb

Min delay:

Name: lib_min

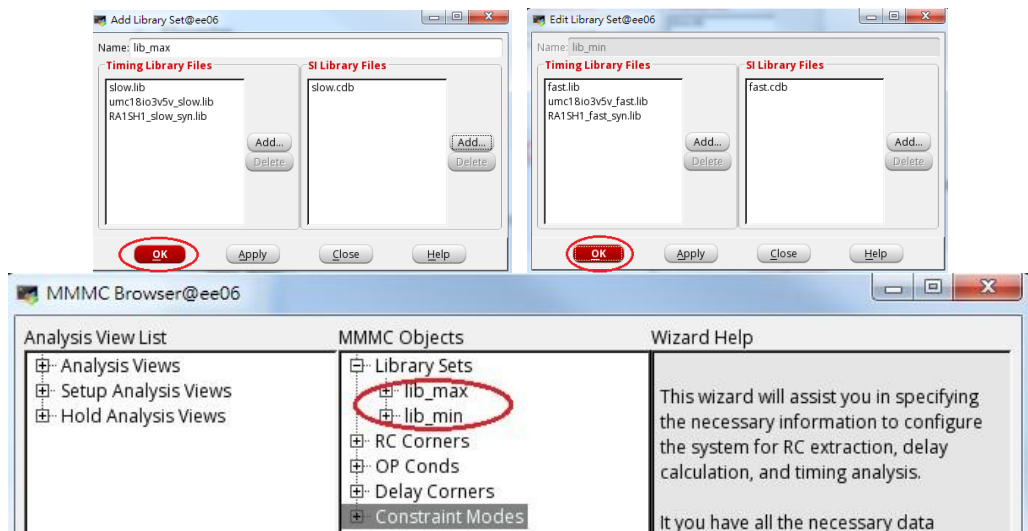
Timing Library:

fsa0m_a_generic_core_ff1p98vm40c.lib

fsa0m_a_t33_generic_io_ff1p98vm40c.lib

Memory_BC.lib

SI Library: u18_ff.cdb

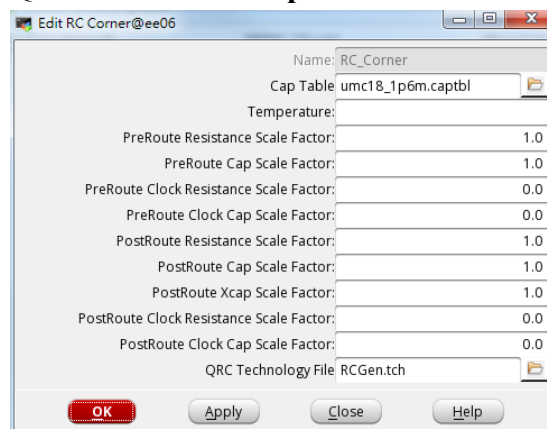


✧ Double click **RC Corners** to include the RC corner library:

Name: RC_Corner

Cap Table: **u18_Faraday.CapTbl**

QRC Tech File: **icecaps.tch**



✧ Double click **Delay Corners** and create max and min delay constraints:

Max delay:

Name: Delay_corner_max

RC Corner: RC_Corner

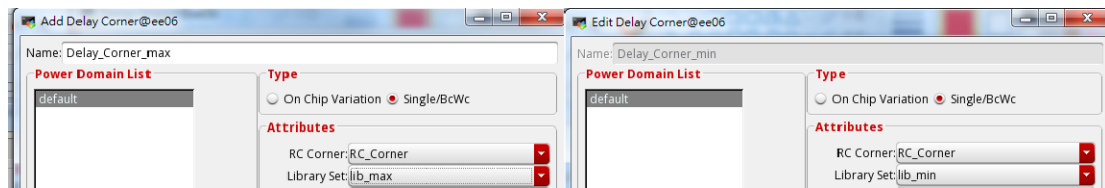
Lib Set: lib_max

Min delay:

Name: Delay_corner_min

RC Corner: RC_Corner

Lib Set: lib_min



- ✧ Double click Constraints Mode and create a function mode to place CHIP.sdc :
Name: func_mode
SDC Constraint Files: CHIP.sdc



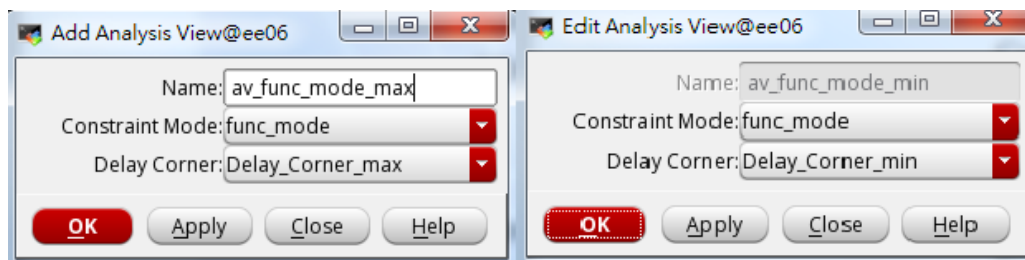
- ✧ Double click **Analysis Views** to create max and min delay analysis

Max delay:

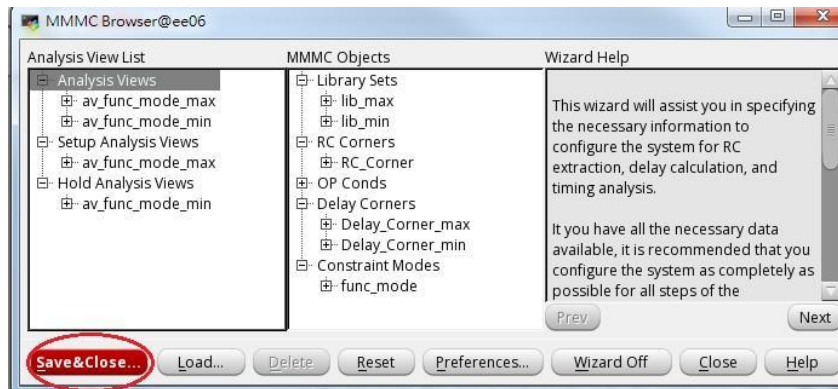
Name: av_func_mode_max
Constraint Mode: func_mode
Delay Corner: Delay_Corner_max

Min delay:

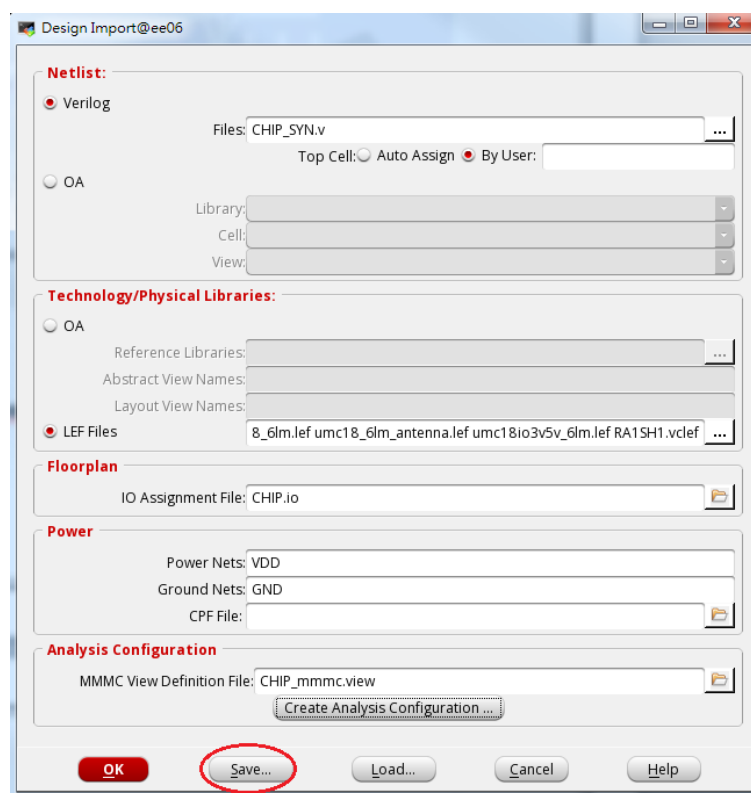
Name: av_func_mode_min
Constraint Mode: func_mode
Delay Corner: Delay_Corner_min



- ✧ Click **Setup Analysis View** and specify the max analysis mode
Choose: av_func_mode_max
- ✧ Click **Hold Analysis View** and specify the min analysis mode
Choose: av_func_mode_min



✧ Save as “CHIP_mmmc.view”



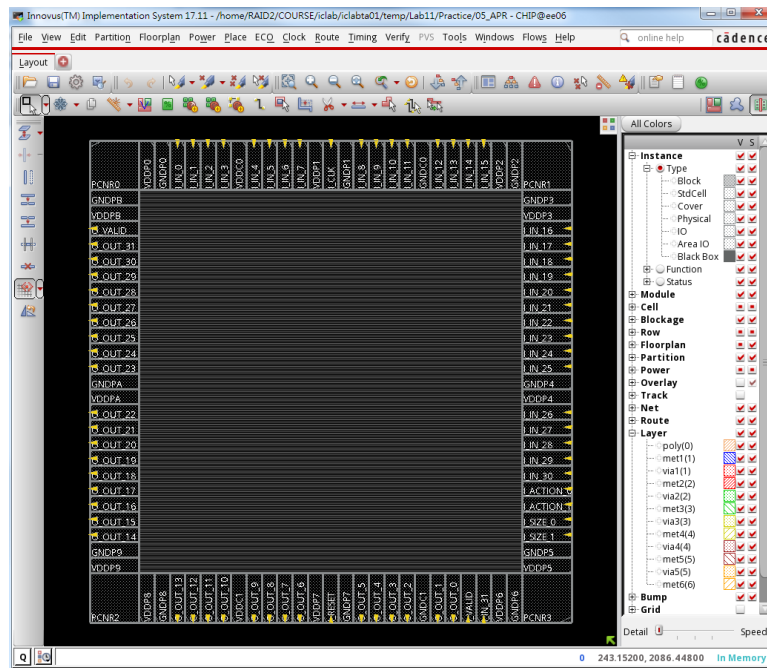
14. Save current settings:

➤ Click **Save...** button

✧ File name: CHIP.globals

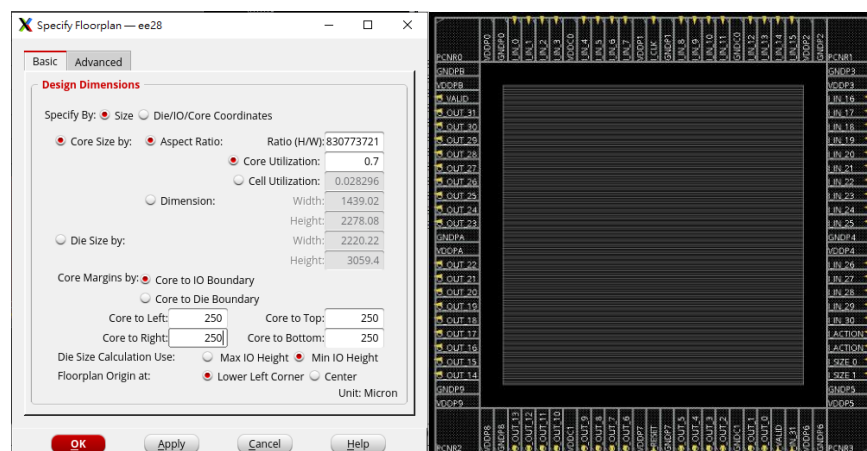
If you want to reload the settings, you can just press **Load...** button

15. Click **OK** button

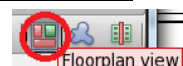


4. Specify Chip Floorplan

16. In innovus menu, open **Floorplan** → **Specify Floorplan**
17. Specify core size:
 - Core Utilization: Between 0 and 1 (Set your desired value) This value will dominate the total chip area
18. Specify core margin:
 - ♦ Core to IO Boundary
 - Core to Left: **250**
 - Core to Right: **250**
 - Core to Top: **250**
 - Core to Bottom: **250**
19. Apply the specification:
 - Click **OK** button



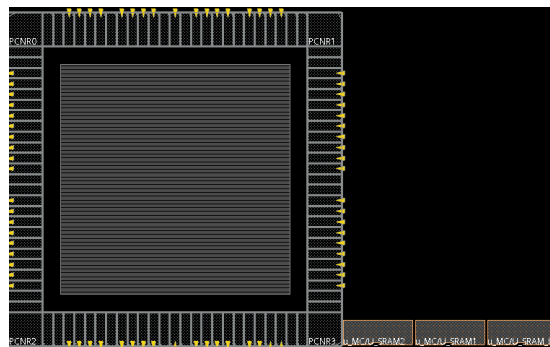
20. You can view designs in three ways; change to floorplan view:



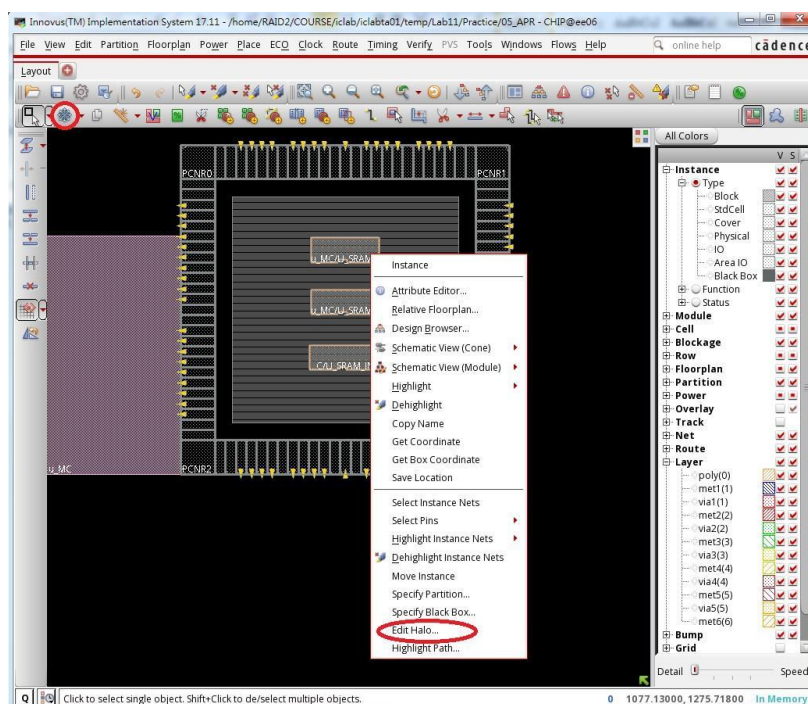
The memory macro should appear (Besides zoom in and zoom out button



, you can use right click to zoom the view)



21. Move them to the desired places with “move” and make placement halo: right click on one macro, choose “Edit Halo”



Both blockage and halo are used to restrict placement or routing according to user-defined rules. The key difference lies in their application:

- Blockage defines a specific region where placement or routing is not allowed.
- Halo defines a keep-out margin around an instance to prevent other cells from being placed or routed nearby.

22. Specify Halo For:

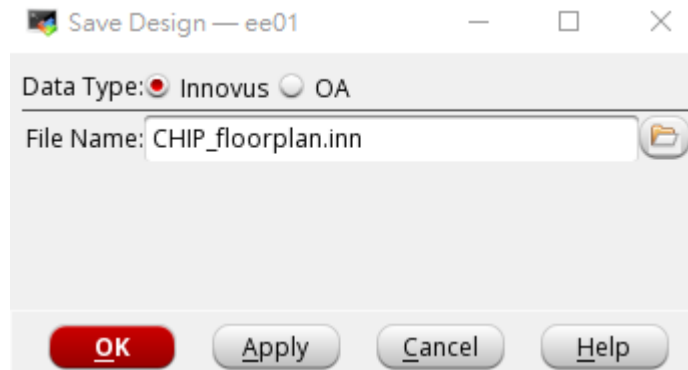
- ◆ All Macros
- Add/Update Halo: Top, Bottom, Left, Right: 15um

23. You can also create the placement blockage yourself with the button:



24. Save the floorplan design:

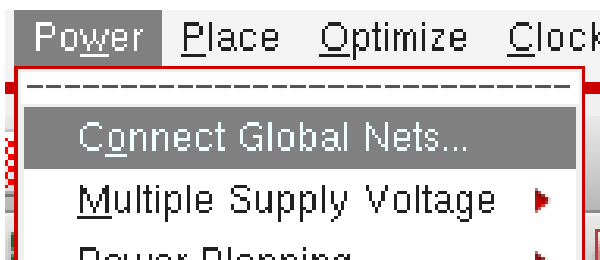
- File → Save Design
 - ✧ Data Type ◆ Innovus
 - ✧ File Name: CHIP_floorplan.inn
 - ✧ Click OK button



If you do the wrong things in the following steps such as power planning, you can restore the design from the previous steps. However, if the input files such as verilog file / io file / timing constraints are changed, you have to rerun all the APR steps.

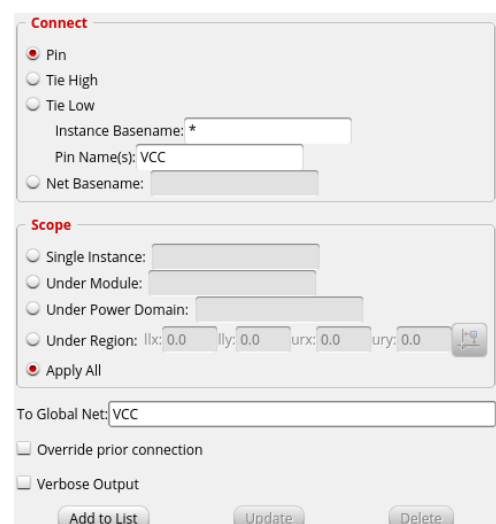
5. Connect / Define Global Net

25. In innovus menu, open **Power** → **Connect Global Nets...**

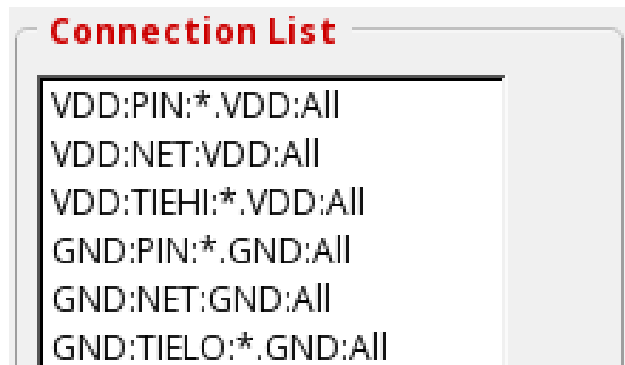


26. Add all VDD pins to Connection List:

- Connect
 - ✧ ◆ Pin
 - ✧ Instance Basename: *
 - ✧ Pin Name(s): VCC
- Scope
 - ✧ ◆ Apply All
- To Global Nets: VCC
- Click **Add to List** button



27. Add all VDD nets to Connection List:
 - Connect
 - ✧ ◆ Net Basename: VCC
 - Scope
 - ✧ ◆ Apply All
 - To Global Nets: VCC
 - Click **Add to List** button
28. Add all Tie High pins to Connection List:
 - Connect
 - ✧ ◆ Tie High
 - Scope
 - ✧ ◆ Apply All
 - To Global Nets: VCC
 - Click **Add to List** button
29. Add all GND pins to Connection List:
 - Connect
 - ✧ ◆ Pin
 - ✧ Instance Basename: *
 - ✧ Pin Name(s): GND
 - Scope
 - ✧ ◆ Apply All
 - To Global Nets: GND
 - Click **Add to List** button
30. Add all GND nets to Connection List:
 - Connect
 - ✧ ◆ Net Basename: GND
 - Scope
 - ✧ ◆ Apply All
 - To Global Nets: GND
 - Click **Add to List** button
31. Add all Tie Low pins to Connection List:
 - Connect
 - ✧ ◆ Tie Low
 - Scope
 - ✧ ◆ Apply All
 - To Global Nets: GND
 - Click **Add to List** button



32. Apply the connection list and check:

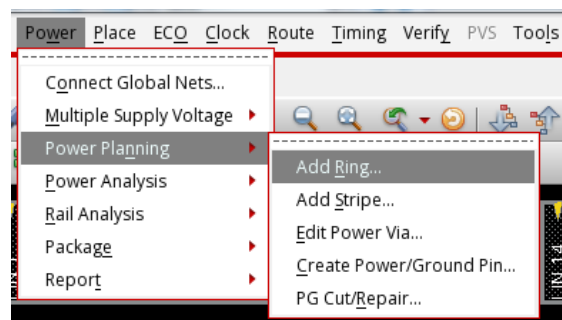
- Click **Apply** button
- Click **Check** button
- Click **Cancel** button

```
Warning: term PD of inst I_CMD15 is not connect to global special net.
Warning: term SMT of inst I_CMD15 is not connect to global special net.
Warning: term PU of inst I_CMD16 is not connect to global special net.
Warning: term PD of inst I_CMD16 is not connect to global special net.
Warning: term SMT of inst I_CMD16 is not connect to global special net.
Warning: term PU of inst I_CMD17 is not connect to global special net.
Warning: term PD of inst I_CMD17 is not connect to global special net.
Warning: term SMT of inst I_CMD17 is not connect to global special net.
Warning: term E of inst 0_BUSY is not connect to global special net.
Warning: term E2 of inst 0_BUSY is not connect to global special net.
Warning: term E4 of inst 0_BUSY is not connect to global special net.
Warning: term E8 of inst 0_BUSY is not connect to global special net.
Warning: term SR of inst 0_BUSY is not connect to global special net.
```

In this lab, warnings only occurring at pads can be ignored.

6. Power Planning (Add Core Power Rings)

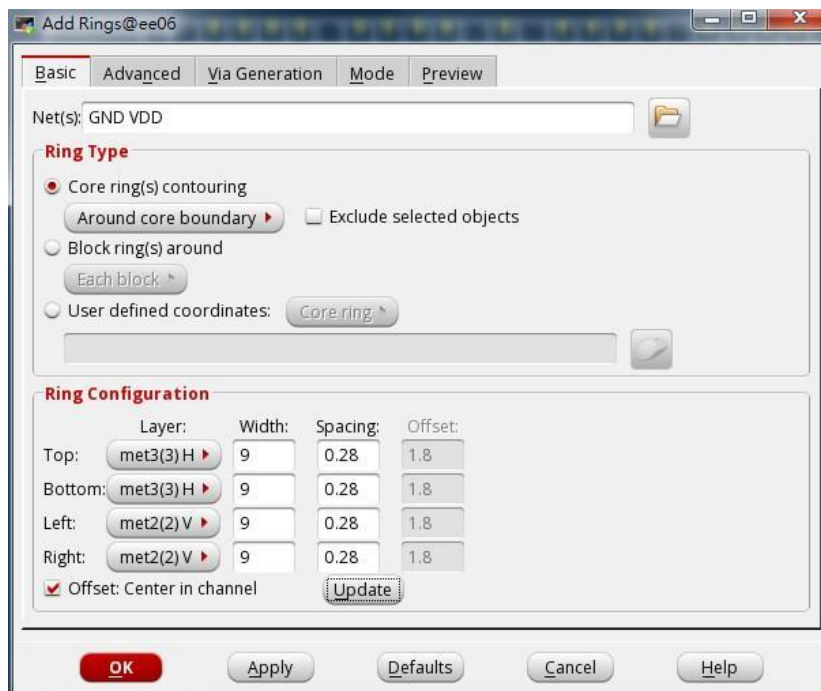
33. In innovus menu, open **Power** → **Power Planning** → **Add Rings...**



34. In the **Basic** tab, fill the following field:

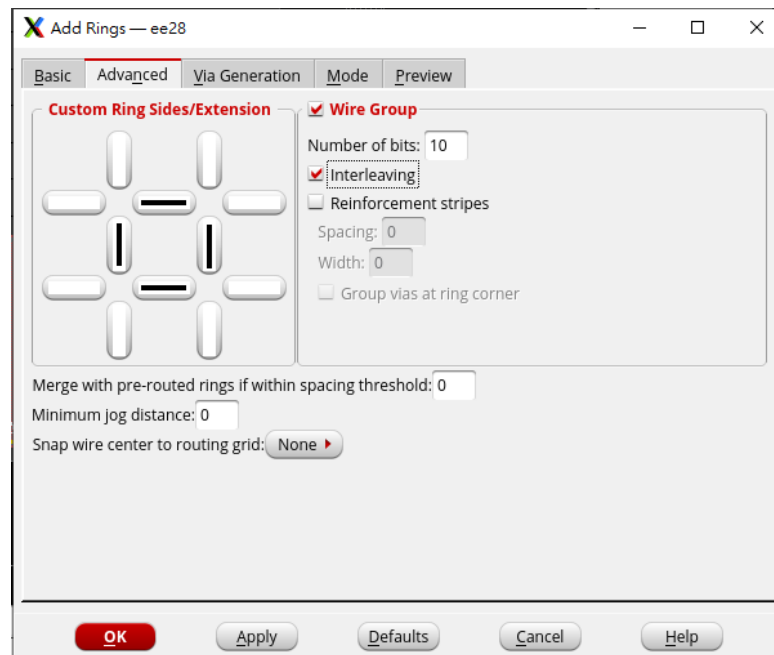
- Net(s): GND VCC
- Ring Type: Core ring(s) contouring
- Specify metal layers and width
 - ✧ Top Layer: **metal3 H** Width: **9**
 - ✧ Bottom Layer: **metal3 H** Width: **9**
 - ✧ Left Layer: **metal2 V** Width: **9**
 - ✧ Right Layer: **metal2 V** Width: **9**

- ✧ ◆ Offset: Center in channel
- ✧ Click **Update** button

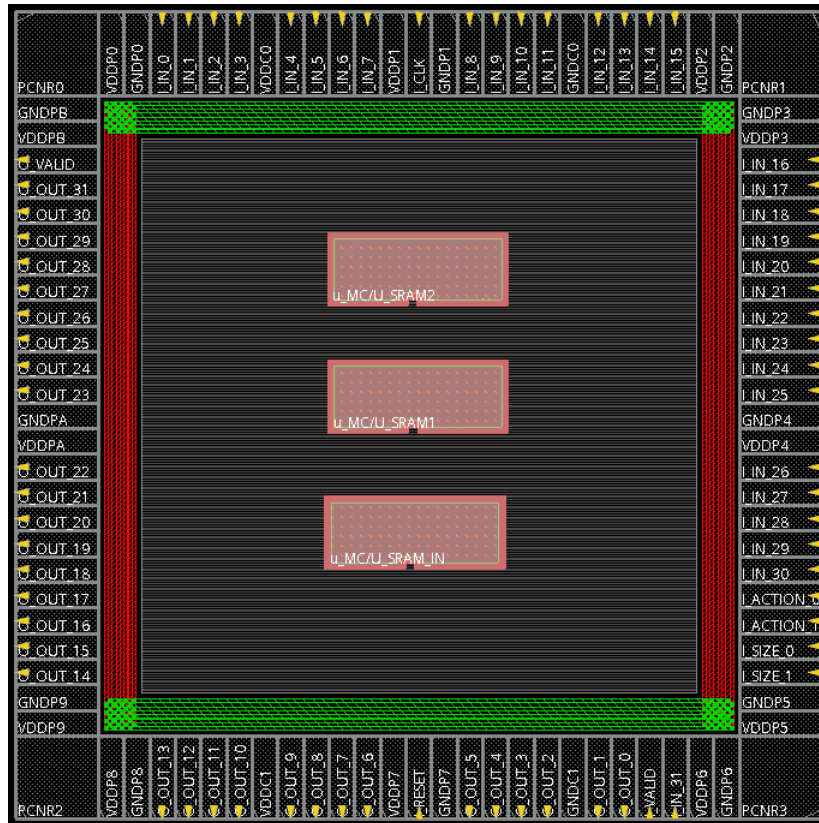


35. In the Advanced tab, fill the following field:

- ◆ Wire Group
- Number of bits: **10**
- ◆ Interleaving
- Click **OK** button

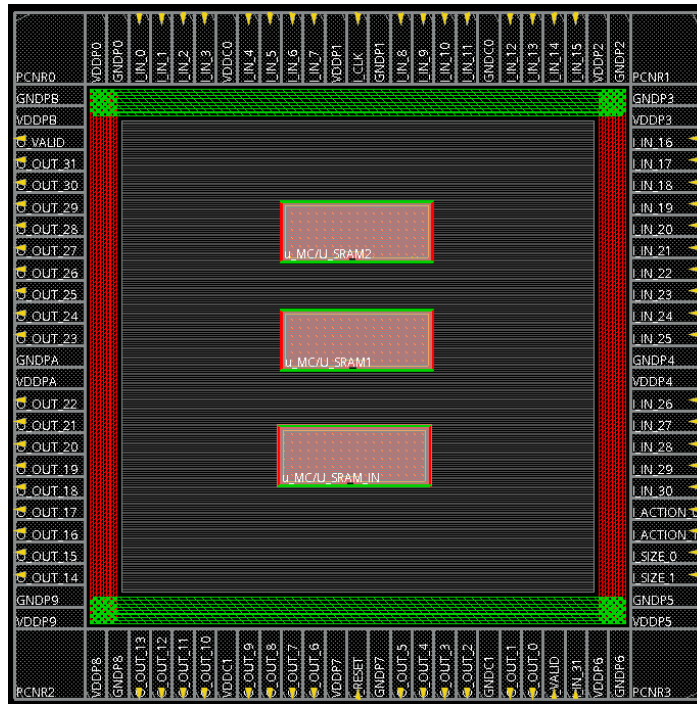


36. Check if the ring is correctly created. If not, click undo button and repeat step 34~35 again.



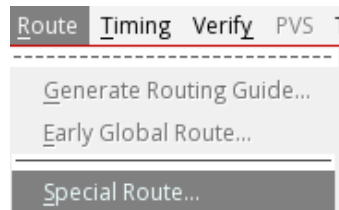
7. Power Planning (Add Block Power Rings)

37. In innovus menu, open **Power** → **Power Planning** → **Add Rings...**
38. In the **Basic** tab, fill the following field:
 - Net(s): GND VCC
 - **Ring Type: Block ring(s) around**
 - Specify metal layers and width
 - ✧ Top Layer: **met3(3) H** Width: 2
 - ✧ Bottom Layer: **met3(3) H** Width: 2
 - ✧ Left Layer: **met2(2) V** Width: 2
 - ✧ Right Layer: **met2(2) V** Width: 2
 - ✧ **Offset: Center in channel**
 - ✧ Click **Update** button
39. In the **Advanced** tab, disable the wire group:
 - **Wire Group**
 - Click **OK** button



8. Connect Core Power Pin

40. In innovus menu, open **Route** → **Special Route...**



41. Connect core power:

In **Basic** tab,

- Net(s): GND VCC
- Set the following configuration

SRoute:

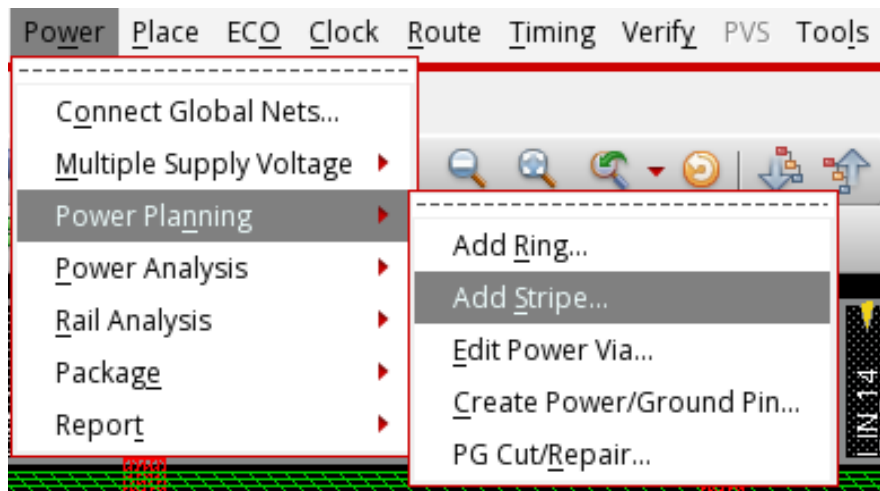
- ✧ ◆ Block pins
- ✧ ◇ Pad rings
- ✧ ◇ Floating Stripes
- ✧ ◆ Pad pins
- ✧ ◇ Follow pins

In **Via Generation** tab,

- Set the following configuration
 - ✧ ◆ Core Ring
 - ✧ ◆ Block Ring
- Click **OK** button

9. Power Planning (Add Stripes)

42. In innovus menu, open **Power** → **Power Planning** → **Add Stripes...**

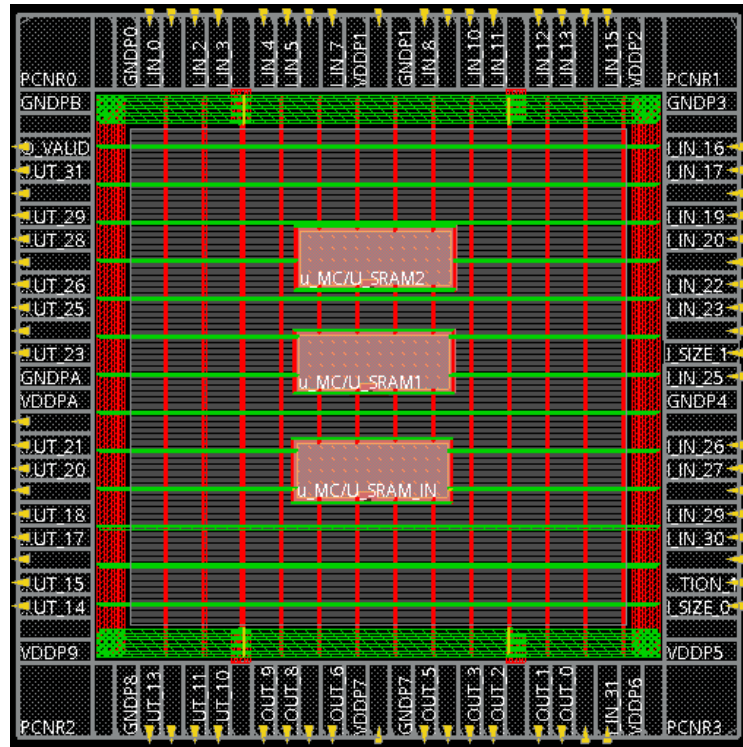


43. Create vertical power stripes:

- Set Configuration
 - ✧ Net(s): GND VCC
 - ✧ Layer: **met2(2)**
 - ✧ Directions: ◆ Vertical
 - ✧ Width: **4**
 - ✧ Click **Update** Button
- Set Pattern
 - ✧ ◆ Set-to-set distance: **100**
- First/Last Stripe
 - ✧ Start from: ◆ Left
 - ✧ ◆ Relative from core or selected area
 - ✧ Start: **50**
- Click **OK** button



44. Check if the stripes are correctly created. If not, click **undo** button  and repeat step 43 again.
45. Create horizontal stripes:
 - Set Configuration
 - ✧ Net(s): GND VCC
 - ✧ Layer: **met3(3)**
 - ✧ Directions: **Horizontal**
 - ✧ Width: **4**
 - ✧ Click **Update** Button
 - Set Pattern
 - ✧ **Set-to-set distance: 100**
 - First/Last Stripe
 - ✧ Start from: **Bottom**
 - ✧ **Relative from core or selected area**
 - ✧ Start: **50**
 - Click **OK** button
46. Check if the stripes are correctly created. If not, click **undo** button  and repeat step 45 again.



10. Connect Standard Cell Power Line

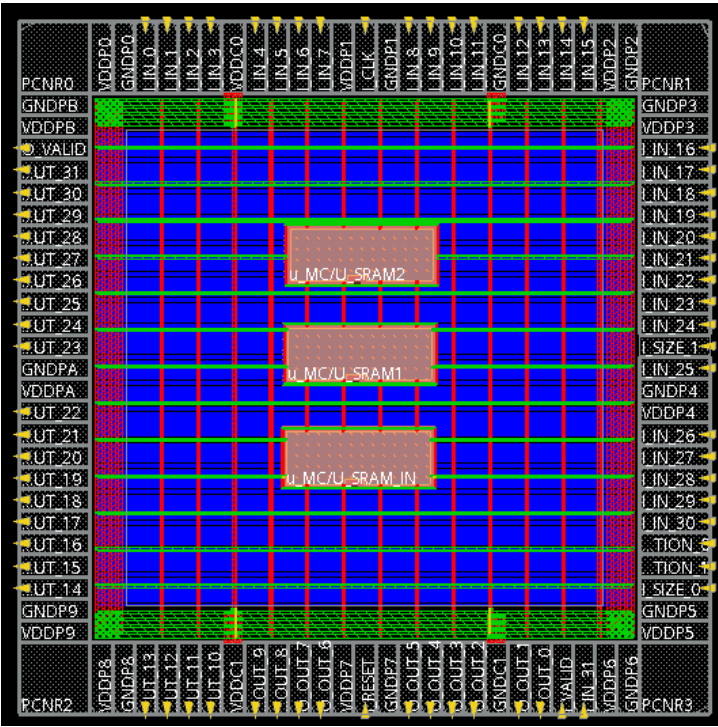
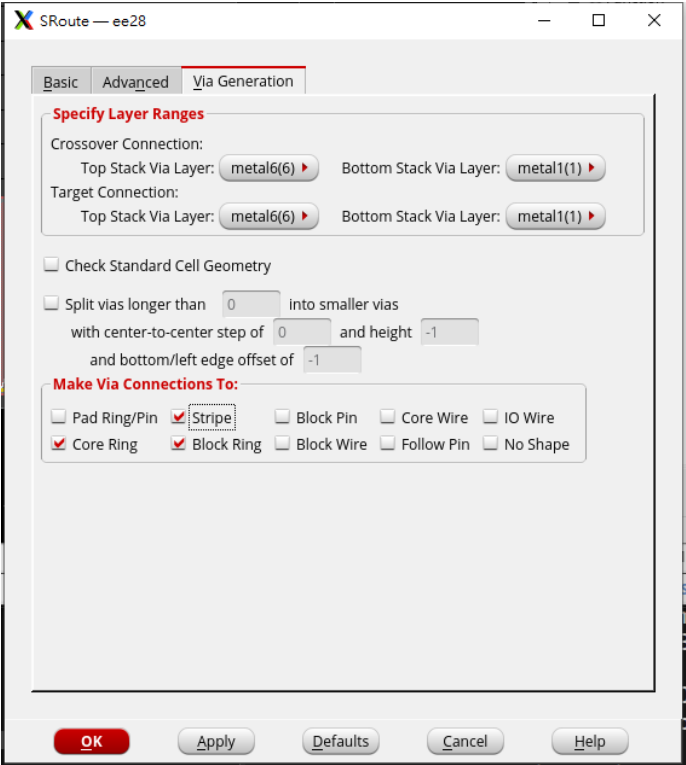
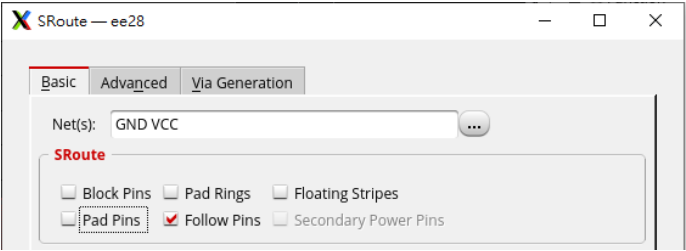
47. Make sure hard macro blockage has been added in floorplan.
48. In innovus menu, open **Route** → **Special Route...**
49. Connect core power:

In **Basic** tab,

- Net(s): GND VCC
- Set the following configuration
 - ✧ ◇ Block pins
 - ✧ ◇ Pad rings
 - ✧ ◇ Floating Stripes
 - ✧ ◇ Pad pins
 - ✧ ◆ Follow pins

In **Via Generation** tab,

- Set the following configuration
 - ✧ ◆ Stripe
 - ✧ ◆ Core Ring
 - ✧ ◆ Block Ring
- Click **OK** button



11. Verify DRC and LVS

50. In innovus menu, open *Verify* → *DRC*

- Click **OK** button
- Check routing for DRC error

```
Verification Complete : 0 Viols.  
*** End Verify DRC (CPU: 0:00:00.3 ELAPSED TIME: 0.00 MEM: 1.0M) ***
```

51. If there are any DRC violations, you can use the following command to fix partial violations:

- fixVia -short

```
innovus 2> fixVia -short
```

- fixVia -minCut

```
innovus 2> fixVia -minCut
```

- Use *Tools* → *Violation Browser* to help finding Violations locations.

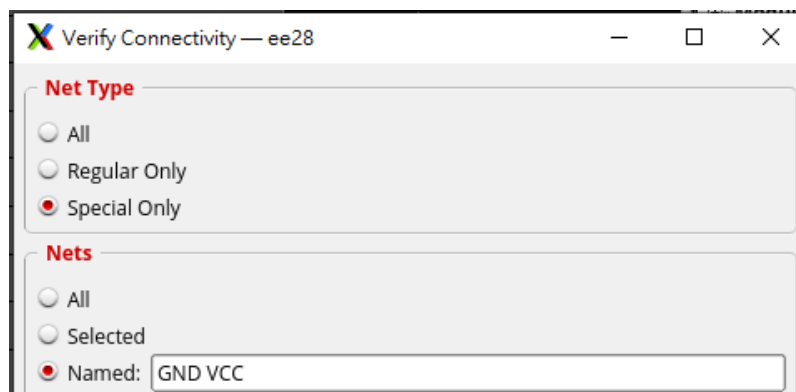
Ex.

Cut_Short via4 → delete the via4 which make this violation

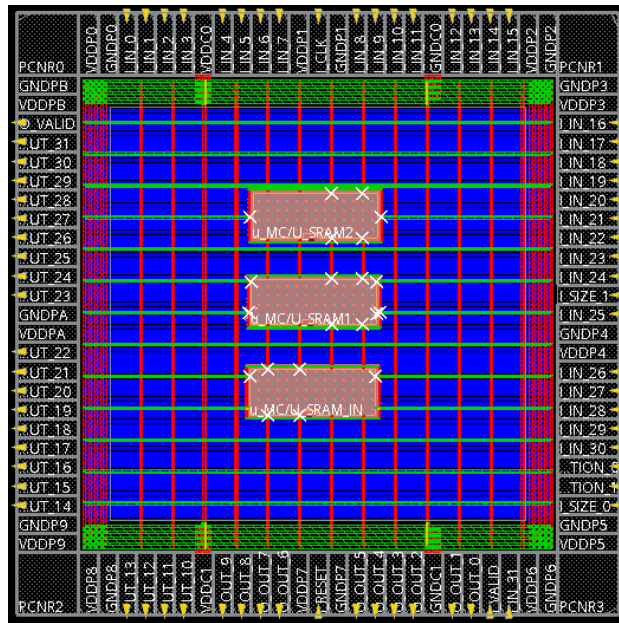
Cut_Spacing via4 → delete the via4 which make this violation

52. In innovus menu, open *Verify* → *Connectivity*

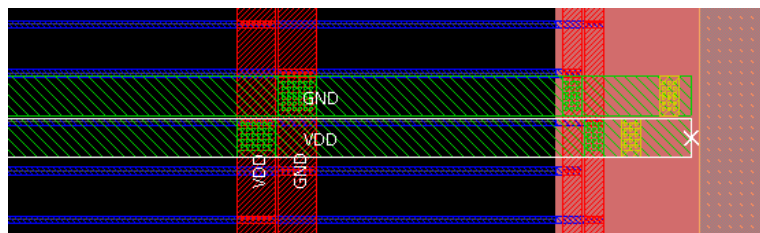
- Net Type: ♦ Special Only
- Nets: ♦ Named: GND VCC
- Click **OK** button
- Check routing for LVS error



53. If the problem is the dangling wire (floating wire) around memory, like:

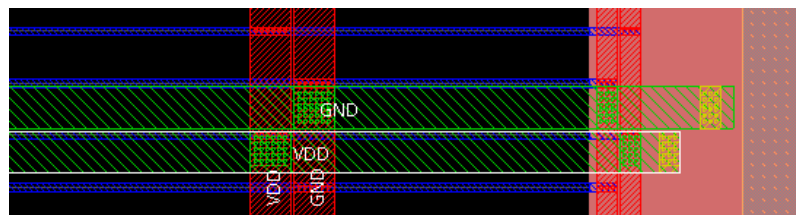


Zoom in and left click to select the highlighted wire:



(In innovus menu, **Tools** → **Violation Browser** can help finding locations)

Press “shift+t” , the dangling wire will be adjusted. (Do not delete those wires or else the power cannot be averagely distributed on the chip)



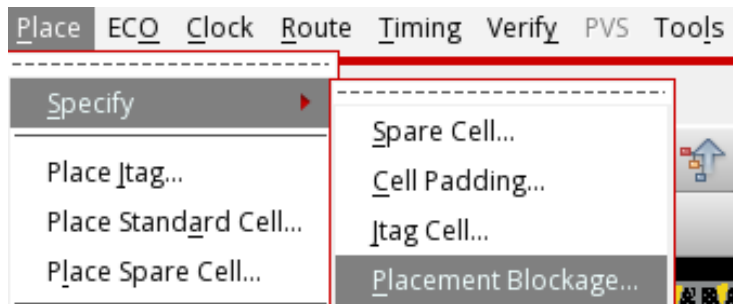
54. After fixing all the dangling wires, run step 52 again to ensure the connectivity correctness.

```
***** End: VERIFY CONNECTIVITY *****
Verification Complete : 0 Viols. 0 Wrngs.
(CPU Time: 0:00:00.0 MEM: 0.000M)
```

55. Save design as CHIP_powerplan.inn

12. Place Standard Cells

56. In innovus menu, open **Place** → **Specify** → **Placement Blockage**



57. Specify placement blockage for stripes

- Specify placement blockage under metal2 and metal3

- ✧ M1

- ✧ M2

- ✧ M3

- ✧ M4

- ✧ M5

- ✧ M6

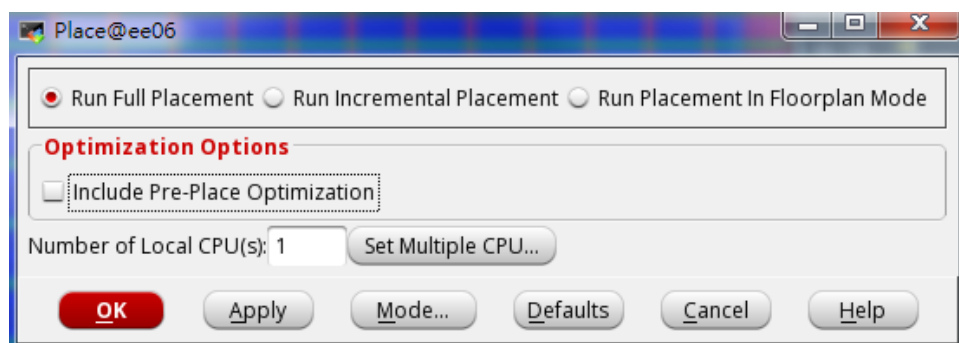
In this step, you should select **all the metal layer used in power stripes** so that standard cells will not be overlapped on power stripes, which may cause DRC violation or LVS error.

- Click **OK** button

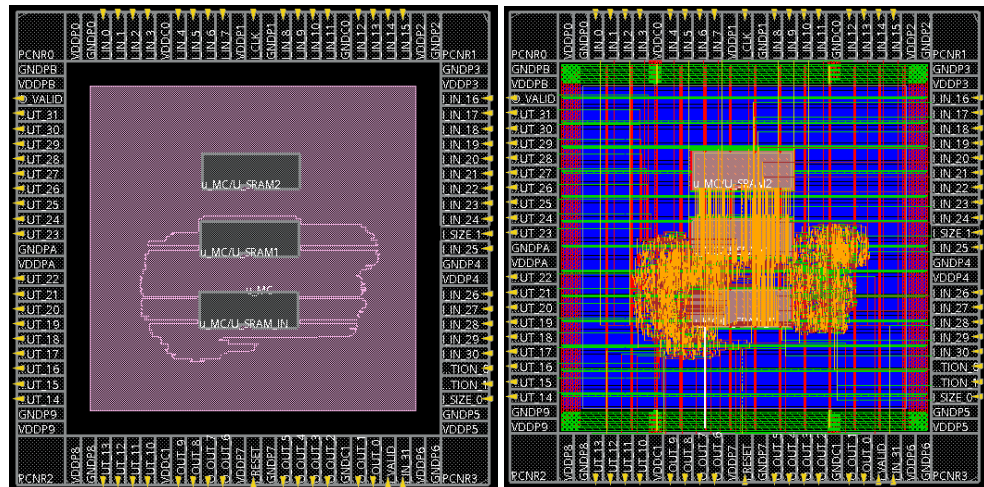


58. In innovus menu, open **Place** → **Place Standard Cells...**

- Run Full Placement
- Optimization Options: ✧ Include Pre-Place Optimization
- Click **OK** button



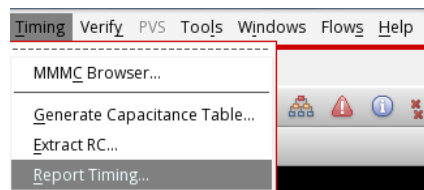
The following figures show the results of Amoeba view and Physical view



59. Save design as CHIP_placement.inn

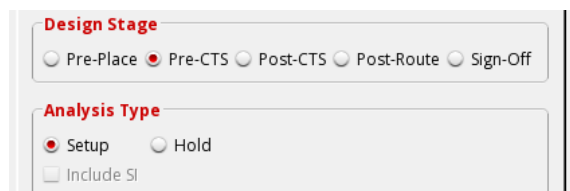
13. In-Place Optimization (IPO) – Before CTS

60. In innovus menu, open **Timing** → **Report Timing...**



61. Perform trial route to model the interconnection RC effects

- Design Stage ◆ pre-CTS
- Analysis Type ◆ Setup
- Click **OK** button



62. See timing reports in **timingReports/** directory, **CHIP_preCTS.slk** shows the timing analysis results. All slack values must be positive value in this file. Moreover, for detail path report, see **CHIP_preCTS_all.tarpt**. DRVs report files: ***.cap**, ***.fanout**, and ***.tran**. You can also see the information shown on the terminal like below.

WNS: Worst negative slack

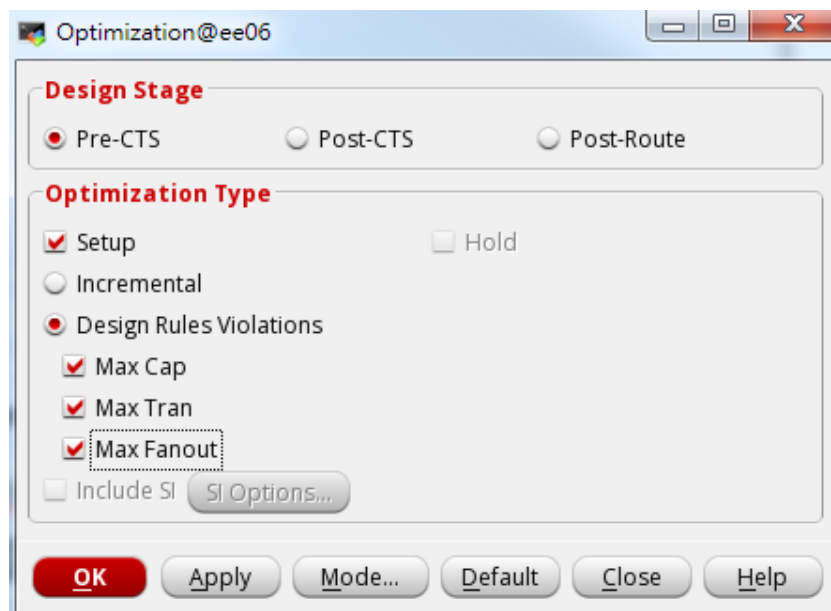
TNS: Total negative slack

Setup mode	all	reg2reg	default
WNS (ns):	-29.113	-18.109	-29.113
TNS (ns):	-45205.9	-19967.7	-25266.8
Violating Paths:	4355	2215	2174
All Paths:	4483	2354	2182

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	2993 (2993)	-3.784	2994 (2994)
max_tran	3801 (64262)	-19.234	3801 (64262)
max_fanout	0 (0)	0	1 (1)
max_length	0 (0)	0	0 (0)

63. If the timing slack is negative, or there are DRVs, open *ECO* → *Optimize Design...* in innovus menu to perform pre-CTS IPO

- Design Stage ♦ Pre-CTS
- Optimization Type
 - ✧ ♦ Setup
 - ✧ ♦ Design Rule Violations:
 - ♦ Max Cap
 - ♦ Max Tran
 - ♦ Max Fanout
- Click **OK** button



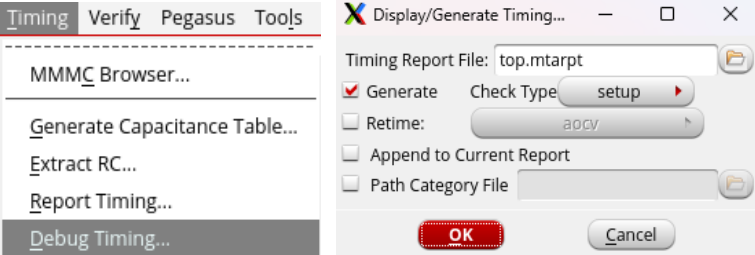
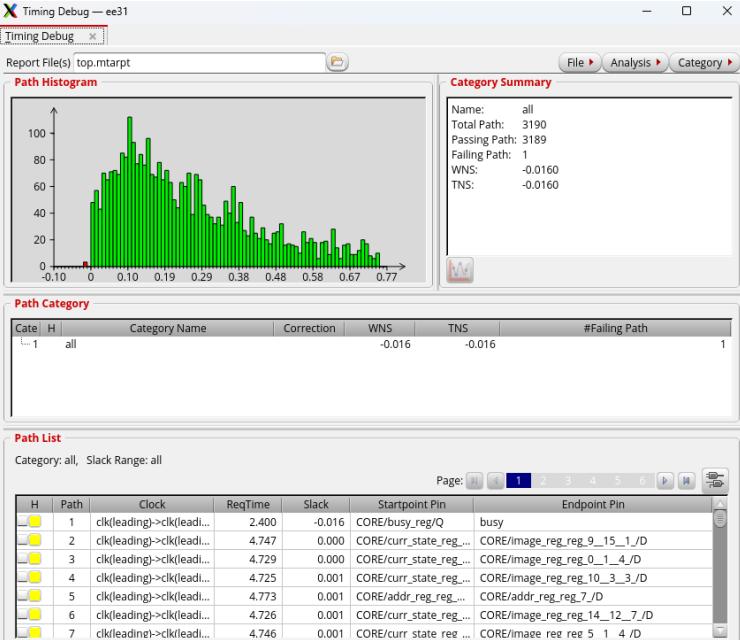
The following figure is the optimized result after pre-CTS IPO.

```

-----
optDesign Final Summary
-----
Setup views included:
av_func_mode_max
-----
+-----+-----+-----+-----+
| Setup mode | all | reg2reg | default |
+-----+-----+-----+-----+
| WNS (ns): | -0.016 | 0.000 | -0.016 |
| TNS (ns): | -0.016 | 0.000 | -0.016 |
| Violating Paths: | 1 | 0 | 1 |
| All Paths: | 4483 | 2354 | 2182 |
+-----+-----+-----+-----+
-----
+-----+-----+-----+-----+
| DRVs | Real | Total |
|-----+-----+-----+-----+
| | Nr nets(terms) | Worst Vio | Nr nets(terms) |
+-----+-----+-----+-----+
| max_cap | 0 (0) | 0.000 | 1 (1) |
| max_tran | 0 (0) | 0.000 | 0 (0) |
| max_fanout | 0 (0) | 0 | 1 (1) |
| max_length | 0 (0) | 0 | 0 (0) |
+-----+-----+-----+-----+
Density: 17.713%
Routing Overflow: 0.02% H and 0.25% V

```

64. You can execute pre-CTS IPO repeatedly until there is no negative slack. However, if the violation paths exist after several pre-CTS IPOs, you can open **Timing → Debug Timing...**, select type: **setup** and click **OK** button.

Timing Debug — ee31

Report File(s) top.mtarpt

Path Histogram

Category Summary

Name: all
Total Path: 3190
Passing Path: 3189
Failing Path: 1
WNS: -0.0160
TNS: -0.0160

Path Category

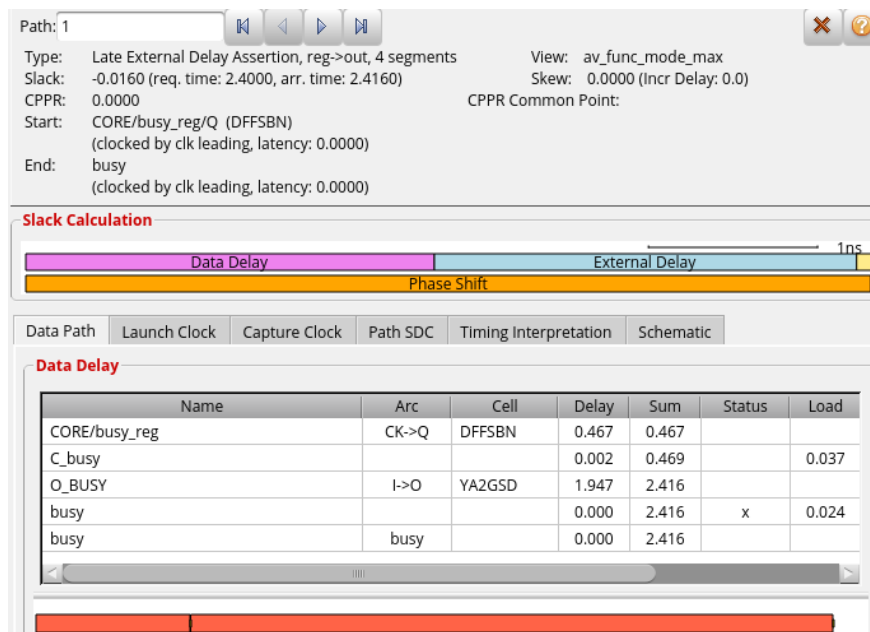
Cate	H	Category Name	Correction	WNS	TNS	#Failing Path
1	all			-0.016	-0.016	1

Path List

Category: all, Slack Range: all

H	Path	Clock	ReqTime	Slack	Startpoint Pin	Endpoint Pin
1	clk(leading)->clk(lead...		2.400	-0.016	CORE/busy_reg/Q	busy
2	clk(leading)->clk(lead...		4.747	0.000	CORE/curr_state_reg...	CORE/image_reg_reg_9_15_1/D
3	clk(leading)->clk(lead...		4.729	0.000	CORE/curr_state_reg...	CORE/image_reg_reg_0_1_4/D
4	clk(leading)->clk(lead...		4.725	0.001	CORE/curr_state_reg...	CORE/image_reg_reg_10_3_3/D
5	clk(leading)->clk(lead...		4.773	0.001	CORE/addr_reg_reg...	CORE/addr_reg_reg_7/D
6	clk(leading)->clk(lead...		4.726	0.001	CORE/curr_state_reg...	CORE/image_reg_reg_14_12_7/D
7	clk(leading)->clk(lead...		4.746	0.001	CORE/curr state reg ...	CORE/maee reg reg 5 1 4 /D

You can double click the violation path and you can easily know why there is setup time violation in this path.



If you think this path cannot be optimized in this step, you can ignore the timing violation since we do not consider the **IO pad delay** during circuit synthesis.

65. The max_cap and max_fanout violations of DRV may be caused by reset_n or clk, which connects to a lot of registers. After ECO, the rst_n DRV may be fixed by inserting buffers. Whereas the clk may not since the clk buffers will be inserted in the next step: Clock Tree Synthesis (CTS) stage. This DRV should be fixed after synthesizing the clock tree, thus don't worry in this stage. You only have to worry if this DRV cannot be fixed after CTS.

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	1 (1)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	1 (1)
max_length	0 (0)	0	0 (0)

66. You will also see **Density** and **Routing Overflow** terms at the bottom of the timing report.

Density: 17.713%
 Routing Overflow: 0.02% H and 0.25% V

- The lower the density is, the lower the standard cells (excluding Hardmacros) are utilizing the core, which means it provides larger flexibility in further routing and optimization steps.
- After the placement and in the pre-CTS stages, innovus performs **trial**

route to give the rough delay calculation of path between every registers / input output port so the pre-CTS can be done. Similar to the density which is the placement utilization, **the routing overflow represents the wiring congestion level. The higher the routing overflow is, the harder of the coming CTS and detail routing (nanoroute).** Usually when Horizontal and Vertical routing overflow are both $< 0.5\% \sim 1.0\%$, the further routing problem will be small. The routing overflow term will disappear after **nanorouting** since the **detail route** will be given to replace the **trial route**.

67. Save design as CHIP_preCTS.inn

14. Clock Tree Synthesis (CTS)

68. Modify the cycle time, input and output delay in **CHIP_cts.sdc**. The cycle time will be used in post-layout simulation (06_POST).

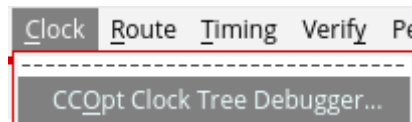
```
=====
# You should modify your desired cycle time for post-layout simulation (06_POST)
# input / output delay should be half of cycle time, or you might get failure in this lab
=====
create_clock [get_ports clk] -period 20.0 -waveform {0 10.0}
set_input_delay 10.0 -clock clk [remove_from_collection [all_inputs] [get_ports clk]]
set_output_delay 10.0 -clock clk [all_outputs]
```

69. In innovus command prompt, execute the following commands:

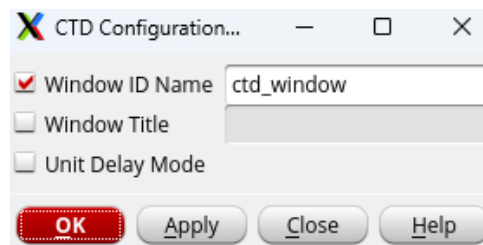
➤ innovus > source ./cmd/ccopt.cmd

```
innovus 3> source ./cmd/ccopt.cmd
```

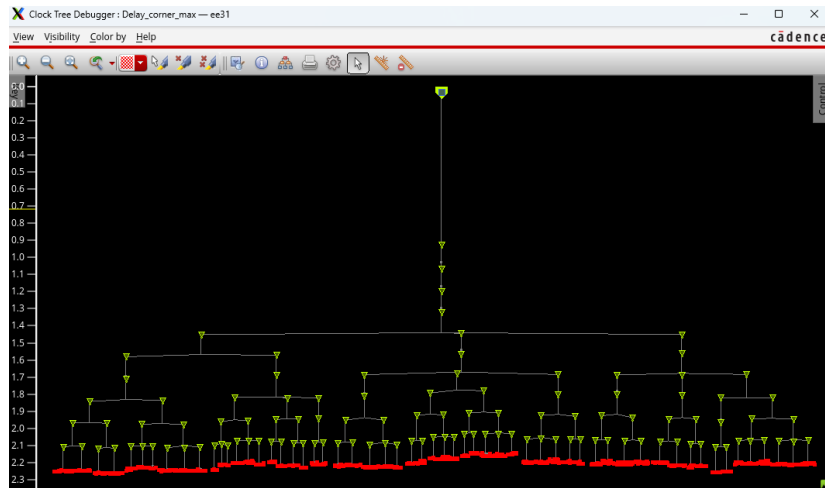
70. If you want to know how the clock tree is synthesized, open **Clock** → **CCOpt Clock Tree Debugger...**



- Window ID Name: ctd_window (default name)
- Click **OK** button

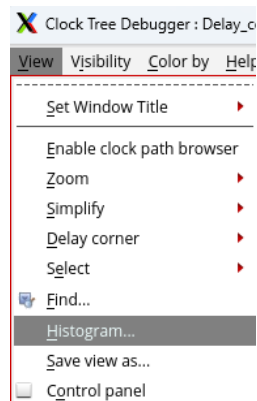


- In Clock Tree Debugger menu, open **View** → **Enable clock path browser**
 - ✧ The inserted clock buffers and their sink DFFs are shown. The clock skew for each skew group is also displayed.

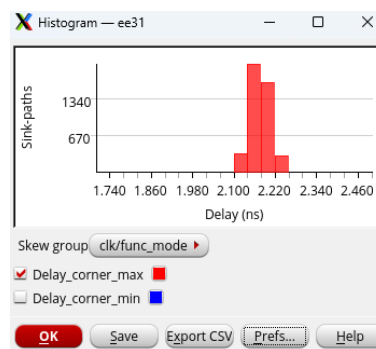


71. Display the histogram of clock network delay

- In Clock Tree Debugger menu, open **View** → **Histogram...**



- In Histogram window, select the following options: **DELAY_MAX**
- In Histogram window, open **Prefs...** (Set Preference)



- In Set Preference window, Histogram Preferences table can adjust the delay range to observe
- Click **OK** button

15. In-Place Optimization (IPO) – After CTS

72. In innovus menu, open **Timing** → **Report Timing...**

Perform trial route to model the interconnection RC effects

- Design Stage ♦ Post-CTS
- Analysis Type ♦ Setup
- Click **OK** button

73. See timing reports in **timingReports/** directory, **CHIP_postCTS.slk** shows the timing analysis results. All slack values must be positive value in this file. Moreover, for detail path report, see **CHIP_postCTS_all.tarpt**. DRV's report files: ***.cap**, ***.fanout**, and ***.tran**. You can also see the information shown on the terminal.

```
Setup views included:
av_func_mode_max
```

Setup mode	all	reg2reg	default
WNS (ns):	-3.211	0.057	-3.211
TNS (ns):	-3.211	0.000	-3.211
Violating Paths:	1	0	1
All Paths:	4483	2354	2182

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 17.814%
Routing **Overflow**: 0.03% H and 0.30% V

74. If the timing slack is negative, or there are DRVs, open **ECO** → **Optimize Design...** in innovus menu to perform post-CTS IPO

- Design Stage ♦ Post-CTS
- Optimization Type
 - ✧ ♦ Setup
 - ✧ ♦ Design Rule Violations:
 - ♦ Max Cap
 - ♦ Max Tran
 - ♦ Max Fanout
- Click OK button

75. **You can execute post-CTS IPO repeatedly until there is no negative slack.** However, if the violation paths exist after several post-CTS IPOs, you can open **Timing** → **Debug Timing...**, select type: **setup** and click **OK** button to know why there are setup time violations. **If you think this path cannot be optimized in this step, you can restore the design: CHIP_preCTS.inn and**

repeat the step 68~69 to re-synthesize the clock tree with larger cycle time set in **CHIP_cts.sdc**.

76. From post-CTS steps, hold time checking is also required. In innovus menu, open **Timing** → **Report Timing...**
77. Perform trial route to model the interconnection RC effects
 - Design Stage ◆ Post-CTS
 - Analysis Type ◆ Hold
 - Click **OK** button
78. See timing reports in **timingReports/** directory, **CHIP_postCTS_hold.slk** shows the timing analysis results. All slack values must be positive value in this file. Moreover, for detail path report, see **CHIP_postCTS_hold_all.tarpt**. DRVs report files: ***.cap**, ***.fanout**, and ***.tran**. You can also see the information shown on the terminal.
79. If the timing slack is negative, or there are DRVs, open **ECO** → **Optimize Design...** in innovus menu to perform post-CTS IPO
 - Design Stage ◆ Post-CTS
 - Optimization Type
 - ✧ ◆ Hold
 - ✧ ◆ Design Rule Violations:
 - ◆ Max Cap
 - ◆ Max Tran
 - ◆ Max Fanout
 - Click **OK** button
80. Save design as **CHIP_postCTS.inn**

16. Add PAD Filler

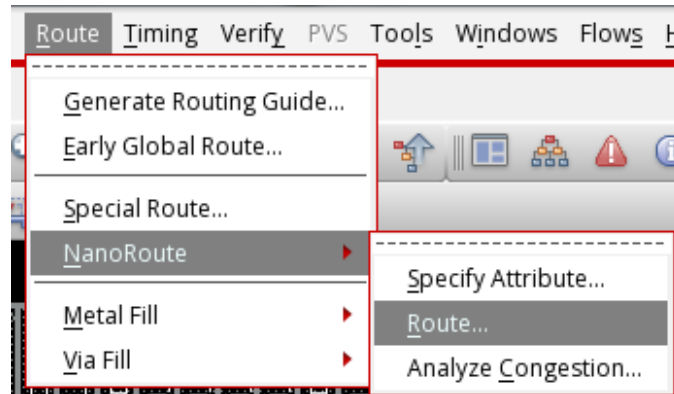
81. In innovus command prompt, execute the following commands:
 - % source ./cmd/addIOFiller.cmd

```
innovus 5> source ./cmd/addIOFiller.cmd
```

-fillAnyGap PAD filler must be added before detail route, or there may have some DRC/LVS violations after PAD filler insertion

17. SI-Prevention Detail Route (NanoRoute)

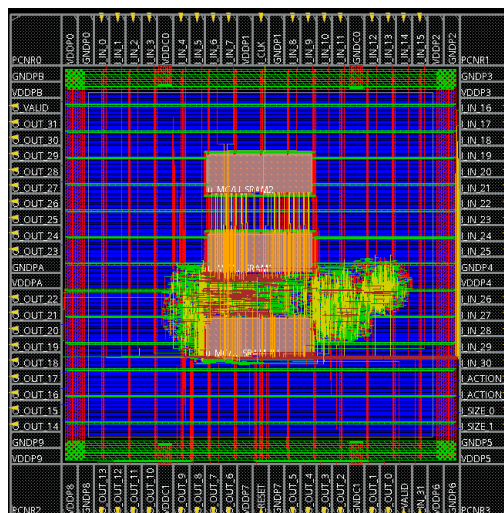
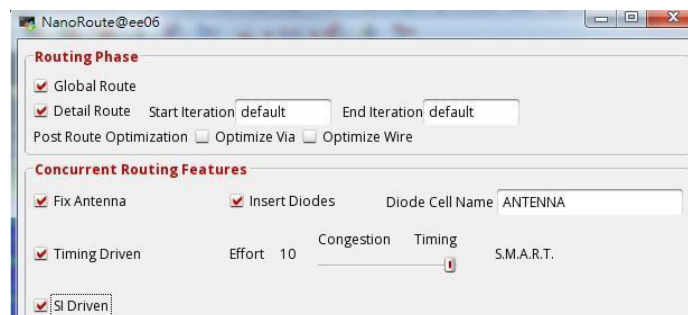
82. In innovus menu, open **Route** → **NanoRoute** → **Route**



83. Nanoroute can prevent cross talk effects and fix antenna rule violations, also it routes design to meet timing constraints.

➤ Configure routing features

- ✧ ◆ Fix Antenna
- ✧ ◆ Insert Diodes Diode Cell Name: **ANTENNA**
- ✧ ◆ Timing Driven Effort: **10**
- ✧ SI Driven
- ✧ Click **OK** button



84. In innovus menu, open **Verify → Connectivity**

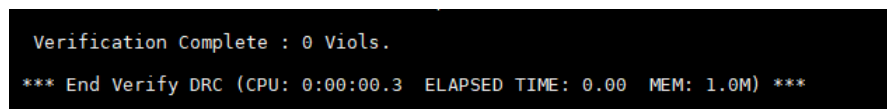
- Net Type: ◆ All
- Nets: ◆ All

- Click **OK** button to check routing for LVS error



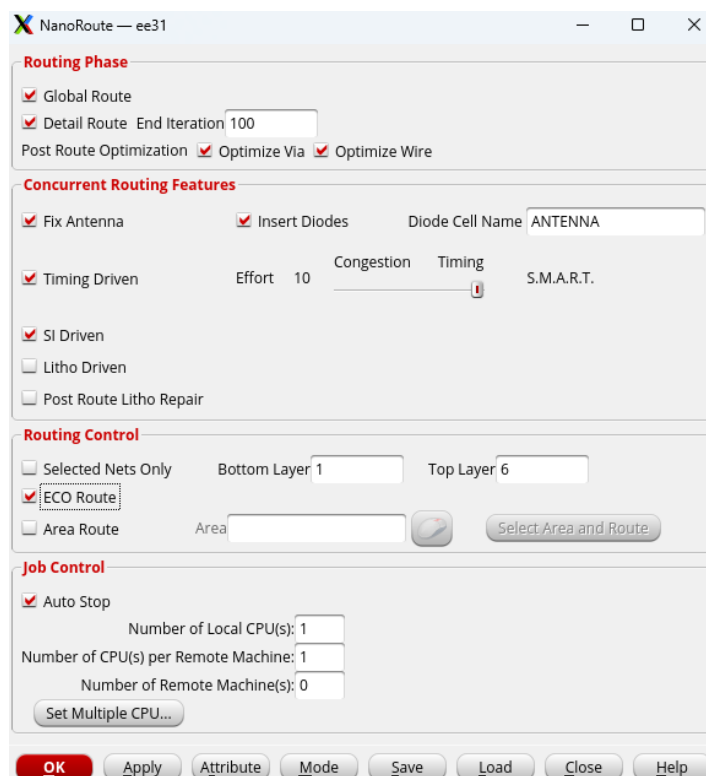
85. In innovus menu, open **Verify** → **DRC**

- Click **OK** button to check routing for DRC violations



- If there are DRC violations, you can open **Route** → **NanoRoute** → **Route** and set Configure routing features as below:

- ✧ **Detail Route** **End Iteration: 100**
- ✧ **Post Route Optimization** **Optimize Via** **Optimize Wire**
- ✧ **Fix Antenna**
- ✧ **Insert Diodes** Diode Cell Name: **ANTENNA**
- ✧ **Timing Driven** Effort: **10**
- ✧ **SI Driven**
- ✧ **ECO Route**
- ✧ Click **OK** button



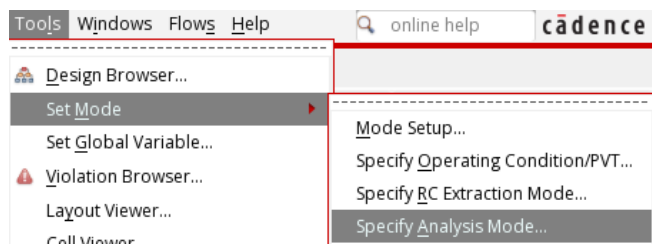
If you still cannot fix DRC violations after executing several nano-route, you should find out the violations and try to fix them with the following tools. **We strongly suggest that you should restart from floorplan and change several parameters if there are hundreds of DRC violations in your chip.**



86. Save design as CHIP_nanoRoute.inn

18. In-Place Optimization (IPO) – Consider crosstalk effects

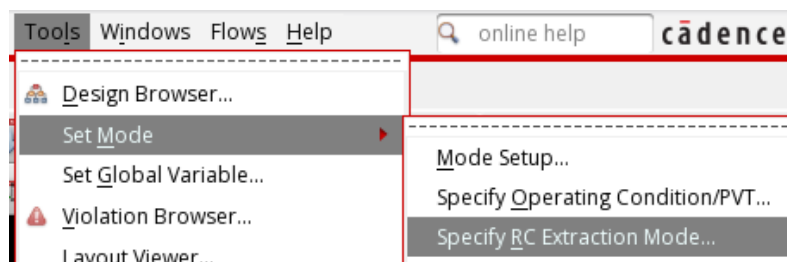
87. In innovus menu, open **Tools** → **Set Mode** → **Specify Analysis Mode...**



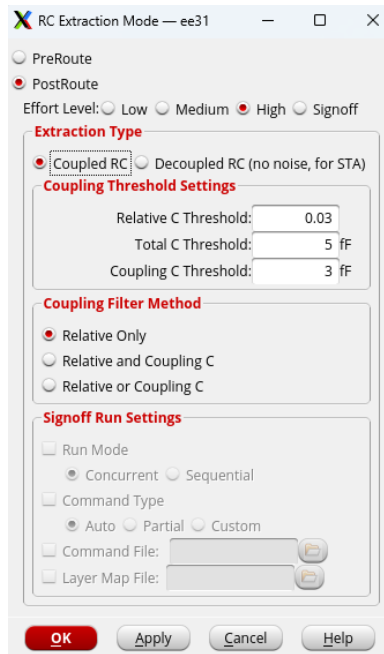
- Timing Mode: ◆ On-Chip Variation
- ◆ CPPR (Clock Path Pessimism Removal)
- Click **OK** button



88. In innovus menu, open **Tools** → **Set Mode** → **Specify RC Extraction Mode...**



- ◆ PostRoute
- Effort Level: ◆ High
- Extraction Type: ◆ Coupled RC
- Coupling Filter Method: ◆ Relative Only
- Click **OK** button



89. Setting rc and si
 - innovus > set_db delaycal_enable_si true
90. In innovus menu, open **Timing** → **Report Timing...**

Perform trial route to model the interconnection RC effects

 - Design Stage ♦ Post-Route
 - Analysis Type ♦ Setup
 - Click **OK** button
91. See timing reports in **timingReports/** directory, **CHIP_postRoute.slk** shows the timing analysis results. All slack values must be positive value in this file. Moreover, for detail path report, see **CHIP_postRoute_all.tarpt**. DRV's report files: ***.cap**, ***.fanout**, and ***.tran**. You can also see the information shown on the terminal.
92. If the timing slack is negative, or there are DRVs, open **ECO** → **Optimize Design...** in innovus menu to perform post-Route IPO
 - Design Stage ♦ Post-Route
 - Optimization Type
 - ✧ ♦ Setup
 - ✧ ♦ Design Rule Violations:
 - ♦ Max Cap
 - ♦ Max Tran
 - ♦ Max Fanout
 - Click **OK** button
 - Repeat this step if there are negative timing slacks
93. In innovus menu, open **Timing** → **Report Timing...**

- Perform trial route to model the interconnection RC effects
- Design Stage ♦ Post-Route
 - Analysis Type ♦ Hold
 - Click **OK** button
94. See timing reports in **timingReports/** directory, **CHIP_postRoute_hold.slk** shows the timing analysis results. All slack values must be positive value in this file. Moreover, for detail path report, see **CHIP_postRoute_all_hold.tarpt**. DRVs report files: *.cap, *.fanout, and *.tran. You can also see the information shown on the terminal.
95. In innovus menu, open **Timing → Report Timing...**
- Perform trial route to model the interconnection RC effects
- Design Stage ♦ Post-Route
 - Analysis Type ♦ Hold
 - Click **OK** button
96. See timing reports in **timingReports/** directory, **CHIP_postRoute_hold.slk** shows the timing analysis results. All slack values must be positive value in this file. Moreover, for detail path report, see **CHIP_postRoute_all_hold.tarpt**. DRVs report files: *.cap, *.fanout, and *.tran. You can also see the information shown on the terminal.
97. If the timing slack is negative, or there are DRVs, open **ECO → Optimize Design...** in innovus menu to perform post-Route IPO
- Design Stage ♦ Post-Route
 - Optimization Type
 - ✧ ♦ Hold
 - ✧ ♦ Design Rule Violations:
 - ♦ Max Cap
 - ♦ Max Tran
 - ♦ Max Fanout
 - Click **OK** button
 - Repeat this step if there are negative timing slacks
98. Save design as CHIP_postRoute.inn

19. Timing Analysis (Sign-off)

99. **You do not execute the sign-off steps in this lab.**
100. Signoff timing analysis is similar to postRoute timing analysis. In addition to invoke Quantus QRC Extraction tool to calculate the RC delay as postRoute, **it also considers the SI affect with the signoff RC.**

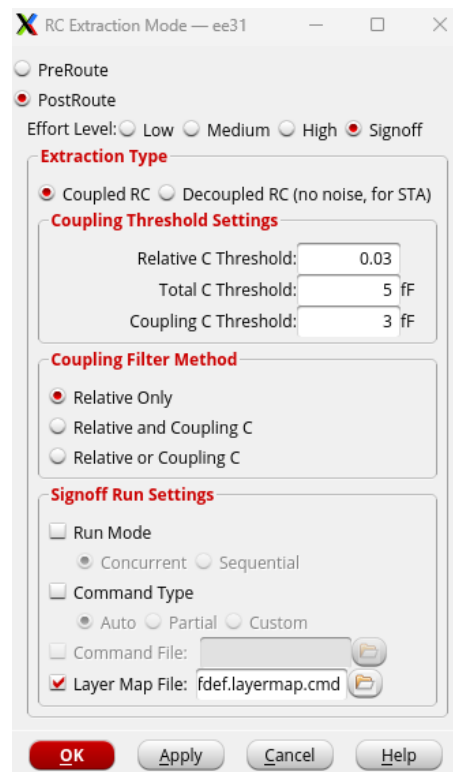
Currently, the delay considering the SI affect with signoff RC can be only

used for timing analysis, but cannot be used for timing optimization. If you want to perform timing analysis or timing optimization for postRoute again after signoff timing analysis, you need to reset the SI mode by the command:

```
innovus > setDelayCalMode -siMode signoff
```

101. In innovus menu, open **Tools** → **Set Mode** → **Specify RC Extraction Mode...**

- ◆ PostRoute
- Effort Level: ◆ Signoff
- Extraction Type: ◆ Coupled RC
- ◆ Layer Map File: lefdef.layermap.cmd
- Click **OK** button



102. In innovus menu, open **Timing** → **Report Timing...**

Perform trial route to model the interconnection RC effects

- Design Stage ◆ Sign-Off
- Analysis Type ◆ Setup
- Click **OK** button

103. See timing reports in **timingReports/** directory, **CHIP_signOff.slk** shows the timing analysis results. All slack values must be positive value in this file. Moreover, for detail path report, see **CHIP_signOff_all.tarpt**. DRVs report files: ***.cap**, ***.fanout**, and ***.tran**. You can also see the information

shown on the terminal.

104. In innovus menu, open **Timing** → **Report Timing...**

Perform trial route to model the interconnection RC effects

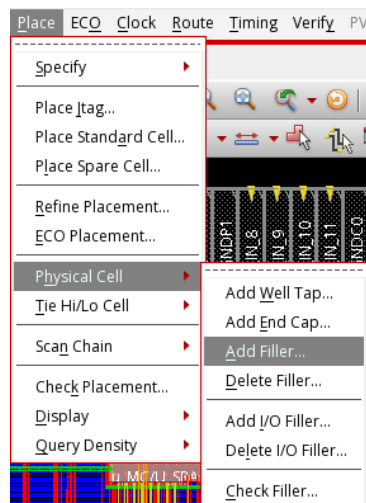
- Design Stage ♦ Sign-Off
- Analysis Type ♦ Hold
- Click **OK** button

105. See timing reports in **timingReports/** directory, **CHIP_signOff_hold.slk** shows the timing analysis results. All slack values must be positive value in this file. Moreover, for detail path report, see

CHIP_signOff_hold_all.tarpt. DRV's report files: *.cap, *.fanout, and *.tran. You can also see the information shown on the terminal.

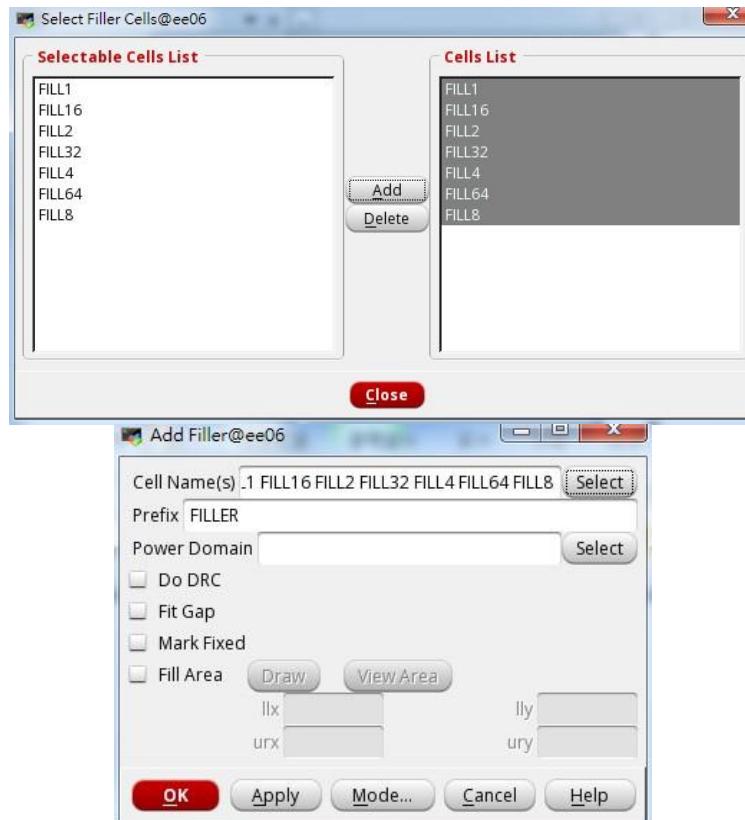
20. Add Core Filler Cells

106. In innovus menu, open **Place** → **Physical Cell** → **Add Filler**



107. Add core filler to improve electric effects of NWELL and PWELL:

- Click **Select** button next to Cell Name(s)
- Select core filler cells
- Click **Add** button
- Click **Close** button
- Click **OK** button



21. Stream Out and Write Netlist

108. DRC check

- Execute **Verify** → **Verify DRC...**
- Check there is no DRC violations in your chip

109. LVS check

- Execute **Verify** → **Verify Connectivity...**
- Check there is no LVS errors in your chip

110. Timing check for postRoute

- Check there is no timing violations

111. Save design as CHIP.inn

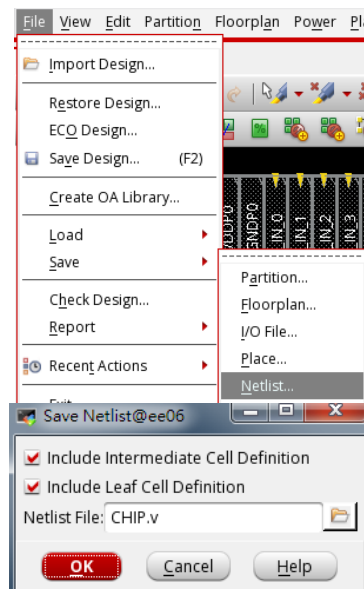
112. In innovus menu, open **Timing** → **Write SDF** to write **CHIP.sdf**

- Delay Calculation Option
 - ✧ ✧ Ideal Clock (Ideal clock should be disabled)
- Click **OK** button



113. In innovus menu, open **File** → **Save** → **Netlist...** to save design netlist CHIP.v for post-layout simulation

- ◆ Include Intermediate Cell Definition
- ◆ Include Leaf Cell Definition
- Netlist File: CHIP.v
- Click **OK** button



22. Post-Layout Gate-Level Simulation

114. Change to directory **06_POST**

115. Perform Post-Layout Gate-level simulation of CHIP.v

% ./01_run_vcs_post

(Note that you should set cycle time the same as that for postRoute, and the latency should be the same as gate level simulation)